EngineCore: An Object-Oriented Simulation of Hybrid Vehicle's Engine's functionality Transformation



Noushien Zahan Neha
ID: 2512877042

Project report for CSE215(Programming language -||)

North South University , Summer '2026

Vehicle System Transformation Simulation

Project Report

Course: CSE215 (Object-Oriented Programming)
Prepared by: Noushien Zahan Neha , ID : 2512877042
Date: August 31, 2026

---

1. Abstract

The Vehicle Transformation Simulation is a desktop application built in Java with a JavaFX graphical interface. It models a vehicle whose engine can operate as manual (combustion), electric, or hybrid, and specifically demonstrates a hybrid engine transforming its mode of operation between manual and electric at runtime. The project was built to satisfy eight core object-oriented and software engineering requirements: a graphical user interface, encapsulation, inheritance, polymorphism, an abstract class, an interface, file input/output, and exception handling. Each requirement is implemented as a functional part of the simulation rather than as an isolated demonstration, so the design choices below are explained in terms of what they accomplish for the application, not just which keyword they satisfy.

---

2. Introduction

2.1 Objective

To design and implement a Java application that simulates a four-wheeler vehicle whose engine can be transformed from a manual (combustion) system to an electric system, using a hybrid engine module that supports both modes internally.

2.2 Scope

The simulation covers:

- Modeling three engine types (manual, electric, hybrid) with shared and distinct behavior
- A vehicle that can be driven, consuming fuel or battery depending on its engine
- A transformation mechanic allowing a hybrid engine to switch operating mode
- Persistent logging of every simulated trip and transformation event to a file
- A JavaFX interface for interacting with the simulation without writing code

---

3. Requirement-to-Implementation Mapping

	Requirement	Implementation
1	Graphical User Interface (JavaFX)	`VehicleSimulationApp` — window with input field, five buttons, two progress bars, and a scrollable status log
2	Encapsulation	Every class uses private fields with public getters/setters (`Engine`, `Vehicle`, all subclasses)
3	Inheritance	`ManualEngine`, `ElectricEngine`, `HybridEngine` all extend the abstract `Engine` class
4	Polymorphism	`Vehicle.drive()` calls `engine.consumeResource()` without knowing which subclass is plugged in; the correct version runs automatically at runtime
5	Abstract Class	`Engine` — declares `getEngineType()` and `consumeResource()` as abstract, forcing every subclass to define its own behavior
6	Interface	`Rechargeable` — implemented only by `ElectricEngine` and `HybridEngine`, since `ManualEngine` has no concept of charging
7	File I/O	`TripLogger` — appends every drive, mode switch, and charge event to `triplog.txt`, and can read it back
8	Exception Handling	`InsufficientResourceException` — a custom checked exception thrown when a drive request would exceed remaining fuel or battery, caught and displayed gracefully by the GUI

---

4. System Architecture

4.1 Class List

Class	Type	Responsibility
`Engine`	Abstract class	Defines the shared contract and shared state (fuel, battery, horsepower) for every engine type
`ManualEngine`	Concrete class	Combustion engine; consumes fuel only

`ElectricEngine`	Concrete class	Battery-powered engine; consumes and can be recharged
`HybridEngine`	Concrete class	Dual-mode engine; the class responsible for the "transformation" feature
`Rechargeable`	Interface	Contract for any engine that supports charging
`Vehicle`	Concrete class	Owns one `Engine` (composition); exposes `drive()`
`TripLogger`	Concrete class	Handles all file writing/reading for the trip log
`InsufficientResourceException`	Custom exception	Signals that a drive request cannot be completed with current resources
`VehicleSimulationApp`	JavaFX Application	The GUI layer; wires buttons to the model classes above

4.2 Design Rationale

Why `Engine` is abstract. There is no such thing as a generic, type-less engine in the real world — every engine is manual, electric, or hybrid. Making `Engine` abstract enforces this at compile time: `new Engine(...)` is illegal, and every subclass is required to supply its own `getEngineType()` and `consumeResource()` logic rather than silently inheriting a default that might not make sense for it.

Why `HybridEngine` carries the transformation feature. Two designs were considered for representing "transformation": swapping the entire `Engine` object inside `Vehicle`, or keeping one `Engine` object whose internal mode changes. The second was chosen because it more accurately reflects how a real hybrid vehicle works — the physical engine doesn't change, only its operating mode does. `HybridEngine.switchMode()` is the single method responsible for this transformation, and `consumeResource()` branches its formula based on the current mode.

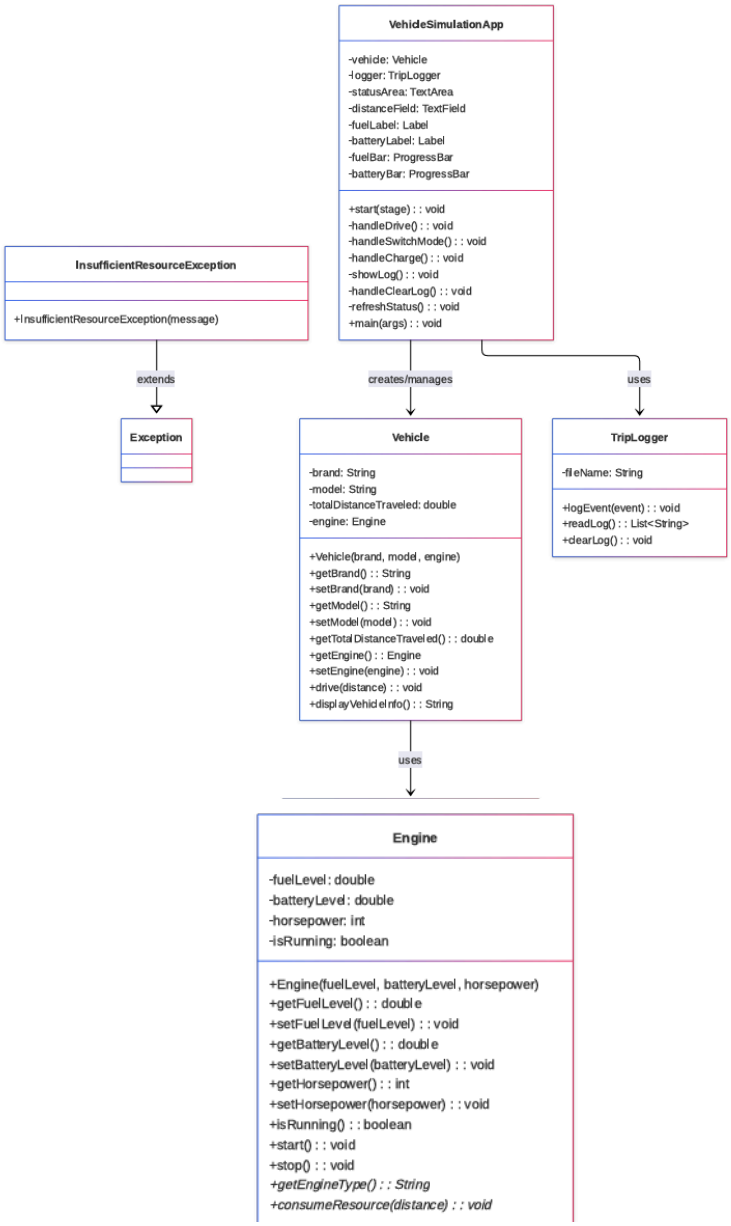
Why `Rechargeable` is an interface rather than part of the `Engine` hierarchy. `ManualEngine` has no meaningful concept of "charging." Placing `charge()` in the abstract `Engine` class would force `ManualEngine` to implement a method that makes no sense for it. Since `ElectricEngine` and `HybridEngine` are not otherwise related to each other by inheritance (they are siblings, both extending `Engine` independently), a shared capability

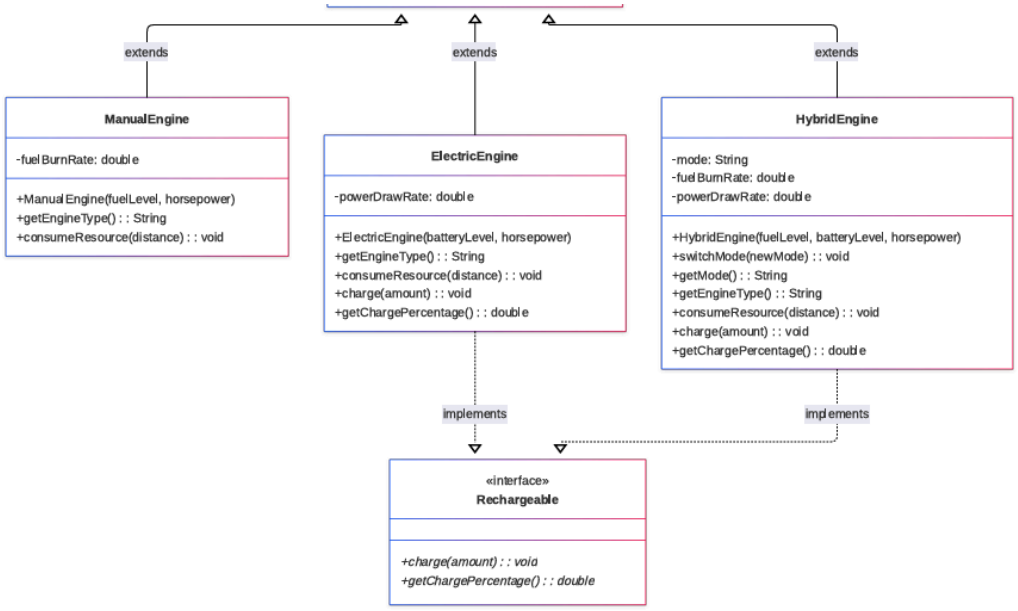
across unrelated branches of the hierarchy is exactly the situation interfaces are designed to solve.

Why `Vehicle` holds an `Engine` by composition rather than inheritance. A vehicle does not become an engine; it has one. `Vehicle`'s constructor accepts any `Engine`, meaning the same `Vehicle` class works unmodified whether it is given a `ManualEngine`, an `ElectricEngine`, or a `HybridEngine` — this is the polymorphism requirement in practice, not just in name.

---

5. UML Diagrams





The diagram above shows the abstract `Engine` class, its three concrete subclasses connected by solid inheritance arrows, and the `Rechargeable` interface connected to `ElectricEngine` and `HybridEngine` by dashed realization arrows — `ManualEngine` is deliberately not connected to `Rechargeable`, since it cannot be charged.

The diagram above titled **vehicle_system_composition_uml** shows how the pieces fit together at the application level: `VehicleSimulationApp` creates a `Vehicle` and writes to a `TripLogger`; `Vehicle` is composed of one `Engine` (marked with a filled diamond, the standard UML notation for composition); and `Vehicle.drive()` can throw `InsufficientResourceException`, which the GUI layer is responsible for catching.

6. Code Reusability and the "Write Once, Run Anywhere" Principle

6.1 Platform Independence (WORA)

When `.java` source files are compiled, `javac` does not produce native machine code for a specific operating system. Instead, it produces **bytecode** — a set of `.class` files in an intermediate format that no processor executes directly. This bytecode is instead run by the **Java Virtual Machine (JVM)**, and a separate JVM implementation exists for Windows, macOS, and Linux. Because the JVM is the layer responsible for translating bytecode into instructions the underlying OS and hardware understand, the exact same compiled `.class` files can run unmodified on any machine with a compatible JVM installed, without needing to be recompiled from source. This is the origin of Sun Microsystems' original slogan: **write once, run anywhere**.

This project is a direct, practical illustration of that principle. The compiled `.class` files for `VehicleSimulationApp` and its supporting classes are not tied to Windows — the same bytecode would run identically on macOS or Linux, provided a matching JVM and the JavaFX runtime modules are available on that machine. This is also precisely why the earlier `"JavaFX runtime components are missing"` error occurred during setup: JavaFX is not bundled inside the JDK itself, so the bytecode was correct and portable, but the *runtime environment* it needed (the JavaFX modules) had to be explicitly supplied via `--module-path` before the JVM could execute it. The bytecode itself never needed to change — only the environment configuration did, which is the WORA guarantee working exactly as intended.

6.2 Code Reusability Through Object-Oriented Design

Separately from platform independence, Java's object-oriented features allow code to be written once and reused across multiple parts of the same program, rather than duplicated. This project relies on that kind of reusability throughout:

Reusable Component	Written Once In	Reused By	Mechanism
`fuelLevel`, `batteryLevel`, getters/setters, `start()`, `stop()`	`Engine`	`ManualEngine`, `ElectricEngine`, `HybridEngine`	Inheritance
`charge()`, `getChargePercentage()` contract	`Rechargeable`	`ElectricEngine`, `HybridEngine`	Interface implementation
`drive()`, `displayVehicleInfo()`	`Vehicle`	Works unmodified regardless of which `Engine` subclass is supplied	Polymorphism
`logEvent()`, `readLog()`, `clearLog()`	`TripLogger`	Every user action in `VehicleSimulationApp`(switch, charge, failed attempts)	Composition

The clearest example is `Vehicle` itself. Its constructor accepts a parameter of type `Engine`, not any specific subclass:

java

```
public Vehicle(String brand, String model, Engine engine) {
    this.engine = engine;
}
```

Because of this, `Vehicle.java` never needed to be rewritten to support three different engine types. The same class works correctly whether it is constructed with a `ManualEngine`, an `ElectricEngine`, or a `HybridEngine` — the differences in behavior are entirely absorbed by polymorphism at the point where `Vehicle.drive()` calls `engine.consumeResource()`. This is code reuse in the strongest sense: not copy-pasted or duplicated with minor edits, but a single implementation that correctly serves every current and future `Engine` subtype without modification.

`TripLogger` demonstrates a second form of reusability: it has no knowledge of vehicles, engines, or driving at all. It simply appends whatever string it is given to a file. This means the exact same class, unmodified, could be dropped into an entirely different application that needed simple event logging — its reusability comes from being deliberately generic rather than tied to this specific domain.

---

6. Exception Handling Strategy

`InsufficientResourceException` is a custom checked exception extending `Exception`. It is thrown from inside `consumeResource()` in each engine subclass whenever a requested drive distance would consume more fuel or battery than currently remains. `Vehicle.drive()` declares `throws InsufficientResourceException` rather than catching it, passing responsibility up to the caller. The GUI's `handleDrive()` method is the point where it is finally caught:

java

```
catch (InsufficientResourceException ex) {
    statusArea.appendText("ERROR: " + ex.getMessage() + "\n");
    logger.logEvent("FAILED drive attempt: " + ex.getMessage());
}
```

This design keeps the exception handling at the layer best equipped to respond to it — the model classes raise the problem, and only the GUI (which controls what the user sees) decides

how to present it. A second `catch (NumberFormatException ex)` block in the same method also guards against non-numeric input in the distance field, so the application cannot crash regardless of what the user types.

---

7. File I/O

`TripLogger` is a dedicated class responsible for all persistence. It uses `FileWriter` in append mode combined with `BufferedWriter` and `PrintWriter` to add a timestamped line to `triplog.txt` after every meaningful action (drive, mode switch, charge, or failed drive attempt). Reading is handled separately through `FileReader` and `BufferedReader`, returning the full log as a `List<String>` for display. Isolating this logic in its own class means the GUI never touches file operations directly — it only calls `logger.logEvent(...)` and `logger.readLog()`, which keeps file-handling concerns separate from interface concerns.

---

8. Graphical User Interface

The interface is built with JavaFX and consists of:

- A text field for entering a drive distance
- Five buttons: Drive, Switch Manual/Electric, Charge Battery, View Trip Log, Clear Log
- Two `ProgressBar` controls showing current fuel and battery levels as a visual percentage
- A scrollable `TextArea` acting as a running status/event log

Each button is bound to a private handler method (`handleDrive()`, `handleSwitchMode()`, `handleCharge()`, `showLog()`, `handleClearLog()`), keeping the `start()` method focused on layout while the actual logic lives in clearly separated methods.

---

9. Sample Execution

A typical session:

1. Application launches with a Tesla Hybrid-X vehicle: 50 L fuel, 100% battery, mode = MANUAL
2. User enters `50` and clicks Drive → fuel drops to 46.0 L (50 km × 0.08 L/km)
3. User clicks Switch Manual/Electric → engine mode becomes ELECTRIC

4. User drives again → battery drops instead of fuel, confirming the transformation took effect
5. User clicks Charge Battery → battery level increases, capped at 100%
6. User attempts an excessively long drive → `InsufficientResourceException` is caught and displayed without crashing the application
7. User clicks View Trip Log → every event above appears with a timestamp, confirming persistence to `triplog.txt`

---

10. Conclusion

The Vehicle Transformation Simulation satisfies all eight required concepts as functioning parts of a coherent application rather than isolated code samples. The abstract `Engine` class and `Rechargeable` interface work together to model real differences between engine types; composition allows `Vehicle` to remain agnostic to which engine it holds; and the custom exception combined with file logging gives the simulation both resilience and a persistent record of activity, all exposed through a JavaFX interface.

11. Future Enhancements

- A `Driver` class to model the person operating the vehicle
- A `TripLog`/`Odometer` class separating distance-tracking logic from `Vehicle`
- Support for constructing multiple vehicles with different engine types simultaneously in the GUI
- Data visualization of fuel/battery usage over time using the existing trip log

---